

SATYAVAANI

Experimental findings: real-time detection of AI voice clones on live calls

Smart India Hackathon · Problem statement SIH26104 · Team-Cognify · updated 26 September 2026

This document collects every experiment we ran while building SATYAVAANI: what each one measured, what it showed, and the design decision it led to. All numbers come from our own runs on our own hardware (NVIDIA RTX 5050 laptop GPU, 8 GB). Where a result comes from a simulation or a logic test rather than real recordings, the text says so.

In progress: two more signals for the risk score. A **scam-language score** (our "sentiment score") will check whether what the caller says sounds like a fraudster, and **pitch and prosody analysis** will check whether the voice moves like a real person's. Section 8 describes both. They are not part of any result in this report yet.

At a glance

What we measured	Result
Deepfake detection, speaker never seen in training (6 teammates, leave-one-speaker-out)	4.6% ± 3.3% EER
... on phone-line (G.711, 8 kHz) audio only	4.7% EER
At the operating threshold	94.7% of clones caught, 7.1% of real voices flagged
Pause-only shortcut check (want ≈ 50%)	10.6% before our fix → 37.5–38.9% after (team); 46.3% with the strict test (LibriSpeech)
Detector trained on one generator, tested on another	40.3% EER (XTTS → kNN-VC), with 5 or 70 training speakers alike
Trained on two generators together (LibriSpeech, 20 unseen speakers)	XTTS 16.1% (unchanged); kNN-VC 40.3% → 24.7% ; both 29.1% → 19.8%
Speaker verification on our team (never trained on)	10.9% (codec) · 11.7% (mic) · 12.1% (mixed) EER: a tie
XTTS clones that pass the voice check	14% on a clean mic → 43% over a phone line
kNN-VC voice-conversion fakes that pass the voice check	71%
Compute per 3 s window, both heads, one encoder pass	18.3 ms (164× faster than real time), 0.28 GB GPU memory

1. What we built and how we tested it

1.1 One encoder pass, two heads

A frozen **wav2vec2-base** encoder (95 M parameters) processes each 3-second window once. All 13 of its hidden layers are kept (L0 is the output right after the convolutional front end, L1–L12 are the transformer layers), and each layer is summarised as the mean and standard deviation over time: 13 × 1,536 numbers per window. Two small heads read the same numbers, each with its own learned softmax weight per layer:

- **Deepfake head** (is this voice synthetic?): layer mix → Linear 1536→64 → ReLU → Linear 64→1, with dropout. An ensemble of three seeds, calibrated to a probability.

- **Speaker head** (is this the person on the caller ID?): layer mix → LayerNorm → Linear 1536→256 → ReLU → Linear 256→192. The output is a 192-number voiceprint, trained with AAM-softmax (margin 0.25, scale 30) on LibriSpeech train-clean-100.
- **Fusion**: risk = probability that the call is synthetic OR not the claimed person, adjusted by call context and smoothed over time (section 5).

Because both heads share one encoder pass, the speaker check is almost free: **2.4 ms for both heads** against 15.9 ms for the encoder.

1.2 Data

Dataset	What it contains	Used for
Our team	6 speakers (Saksham, Asmit, Vijay, Nikhil, Yogya, Shreyas) × 30 phone-recorded sentences (25 English + 5 Hindi), plus a reference passage of about 40 s each	Deepfake training and testing; speaker testing; demo calls
XTTS-v2 clones of the team	180 clones: the same sentence, in the same voice, conditioned on that person's reference passage	The fake class
kNN-VC fakes of the team	120 voice-conversion fakes: another teammate's sentences 011–030 converted into each person's voice, using their reference passage plus sentences 001–010 (about 80 s) as the voice sample	Unseen-generator test
LibriSpeech train-clean-100	251 English audiobook speakers	Speaker-head training (211 train / 40 held out)
LibriSpeech generalisation set	100 speakers × 8 sentences, split 70 train / 10 calibrate / 20 test with no speaker in two groups; 800 XTTS clones; 800 kNN-VC fakes (640 train/calibrate, 160 test)	One vs two generators, at scale

1.3 How we measured

- **EER (equal error rate)**: the error rate at the threshold where "real voice flagged as fake" equals "fake let through". Lower is better; 50% is guessing.
- **Leave-one-speaker-out (6-fold)**: train on five teammates, test on the sixth, rotate. The test person's voice and sentences are never seen in training.
- **Speaker-disjoint splits** on LibriSpeech: training, calibration and test speakers never overlap.
- **Enrol / test split** for voiceprints: sentences 001–010 build a voiceprint; sentences 011–030 are only ever used for testing and demo calls.
- **Identical processing for real and fake**: every change applied to real audio is applied to fakes in exactly the same way, so the processing itself can never give the answer away.
- **Silence-only check**: a head trained on the pauses alone. If pauses carry no clue it scores about 50%; a much lower score means the detector may be cheating on the background.

2. Deepfake head

2.1 A perfect score was a warning sign

Our first prototype compared public real speech (FLEURS) with fakes from different sources (edge-tts voices and Griffin-Lim vocoding) and scored **0% EER**. The number was meaningless: real and fake came from different recording pipelines, so the detector could separate them without listening for synthesis at all. Trained on one fake source and tested on the other, it scored 56.3% and 44.8% EER, around guessing.

Decision: record our own team and clone the exact same sentences in the exact same voices, so the only difference left between the classes is the synthesis itself.

2.2 The pauses gave the clones away, and how we fixed it

The first honest run on the team data looked excellent: **2.5% ± 1.8% EER**. But the silence-only check came out at **10.6%**: a head that heard only the pauses could already tell real from fake. Phone recordings carry room tone in the gaps between words; XTTS pauses are almost digitally silent. Part of the 2.5% was the room, not the voice.

The fix, applied identically to real and fake clips:

- **Silence the gaps between words**, keeping 40 ms around each word with 10 ms fades, then **fill everything with random background noise** in a random colour from hiss to rumble. The model never sees a clean pause in training.
- **Random microphone colouring** (a random spectral tilt), so a microphone's character can't identify the class.
- **Louder background noise**: 5–30 dB SNR instead of 10–35 dB.

Before and after the fix, every clip also had a **phone-line copy** (8 kHz, G.711 μ -law), so half of all training audio sounds like a call.

Silence-only rose from 10.6% to **38.9%**, and speech EER settled at **4.0% ± 2.2%**. The move from 2.5% to 4.0% is the shortcut being removed: 4.0% is the honest number. In a simulation of our situation (room tone vs near-silent pauses), the old recipe gave 9.7% silence-only EER and the new one 46.6%. The speech itself passes through unchanged: only the gaps are replaced.

Why a detector must not rely on pauses in real calls:

- Mobile and VoIP codecs often replace silence with generated comfort noise, so pauses reflect the network, not the caller.
- Attackers add background noise on purpose; it takes one click in any audio tool.
- Genuine callers range from quiet rooms to busy streets. "Clean pauses = fake" would also flag honest people.

2.3 How the recipe evolved (team, 6-fold leave-one-speaker-out)

Training recipe	EER (all)	EER (phone line)	Silence-only (want ≈50%)	Clones caught / real flagged
Background noise (10–35 dB) + phone-line copy	2.5% ± 1.8%	2.1%	10.6% (shortcut)	not valid (calibration bug, 2.5)
+ pause gating, mic tilt, 5–30 dB noise	4.0% ± 2.2%	3.9%	38.9%	93.4% / 8.5%
+ pitch/speed variation (final)	4.6% ± 3.3%	4.7%	37.5%	94.7% / 7.1%

About 1,400 windows of 3 s per run (630 real / 780 fake in the final run). Per-speaker numbers are in Appendix A.

2.4 Pitch and speed variation

When a caller is shocked or excited their pitch rises, and early live tests showed the risk climbing on genuine emotional speech. We added speed perturbation to training: each copy of each clip, real and fake alike, is played 0.90–1.10× as fast. That shifts pitch by about –1.8 to +1.7 semitones (a 200 Hz voice lands between 180 and 220 Hz). The detector learns that a pitch change on its own is not evidence of a clone.

Cost and benefit. EER on calm test speech moved from 4.0% to 4.6%, within noise for six speakers. At the operating threshold it caught more clones (93.4% → 94.7%) and flagged fewer real voices (8.5% → 7.1%). The hardest speaker in the final run is Vijay (11.9% EER), almost all of it on his clean recordings (18.4% clean vs 5.3% phone line). We also tried adding pitch-shifted copies (0.92× and 1.08×) of the enrolment

sentences to the speaker voiceprints; the final voiceprints leave them out and use only the recorded voice through the phone line (section 4.3).

2.5 Probability calibration: a bug worth reporting

The heads output a raw score, and Platt scaling turns it into a probability. Our first implementation diverged on very confident scores (a slope of about 10^7), which made the app report "0% of clones caught". We fixed it by rescaling the scores first and taking damped Newton steps that can only lower the loss. Re-tested on synthetic scores scaled anywhere from ± 1 to ± 300 , it now converges every time and puts the 50% line at the equal-error point. On our data the calibrated threshold flags 7.1% of real voices and catches 94.7% of clones.

2.6 The phone line does not hurt detection

On phone-line copies only, EER is 4.7% against 4.6% overall (3.9% vs 4.0% in the previous recipe). Calls arrive as narrow-band phone audio, so this is the condition that matters, and the detector holds up in it.

2.7 A new generator breaks it: kNN-VC on our team

We generated 120 fakes with **kNN-VC**, a voice-conversion model that is a different kind of attack from XTTS (someone speaks, and it comes out in the target's voice) and a different model family (WavLM features, nearest-neighbour matching, its own vocoder). Each teammate was scored by the head that had never heard them and had only ever seen XTTS fakes.

Test	Result
EER vs XTTS clones (seen generator, same test clips)	4.1%
EER vs kNN-VC (unseen generator)	40.3% $\pm$ 14.4%
kNN-VC fakes caught at the app's threshold	12.9% (real voices flagged: 6.4%)
kNN-VC fakes that pass the voice check	71% (see 4.4)

Per speaker, kNN-VC EER ranged from 18.5% (Nikhil) to 63.6% (Shreyas); see Appendix A.

Finding: a head trained on one generator learns that generator's fingerprint ("what XTTS sounds like"), not what synthetic speech sounds like in general.

2.8 More speakers did not fix it (LibriSpeech, 70 training speakers)

We repeated the test at scale: 100 LibriSpeech speakers, XTTS clones of their own sentences, and kNN-VC fakes for the 20 test speakers. All audio was phone-line, with the same pause gating and noise. Trained on 70 speakers' XTTS clones:

Recipe	EER vs XTTS (seen)	EER vs kNN-VC (unseen)	Real flagged	XTTS / kNN-VC caught
with pitch/speed variation	16.3%	36.6%	15.6%	82.8% / 39.4%
without pitch/speed variation	16.1%	40.3%	13.1%	— / 30.1%

With 70 training speakers instead of 5, the unseen-generator EER stayed near 40%. **Data volume was not the problem.** Pitch variation helped slightly on the unseen generator (36.6% vs 40.3%), but with 20 test speakers that is within noise. LibriSpeech is a harder test than our team for the seen generator too (16% vs 4.6%): its recordings and speakers vary far more.

2.9 More noise made it worse

Background noise	Silence-only	EER vs XTTS	EER vs kNN-VC	kNN-VC caught	Real flagged
5–30 dB (kept)	31.2%	16.1%	40.3%	30.1%	13.1%
0–20 dB (louder)	37.5%	17.7%	45.9%	30.1%	14.8%

Louder noise hides the traces of both generators and teaches nothing that transfers. We kept 5–30 dB. The silence-only check also needed tightening: after gating, the "pause" snippets still held the first and last few milliseconds of words, which are genuine synthesis evidence rather than background. The **strict** version uses only frames at least 100 ms away from any word.

2.10 Two generators trained together

Same LibriSpeech speakers (70 train / 10 calibrate / 20 test), phone-line audio, 5–30 dB noise. We made kNN-VC fakes for the training and calibration speakers too, each speaker's voice taken over by another speaker in the same group, so no test speaker's audio ever reached training. Three heads were trained on identical features:

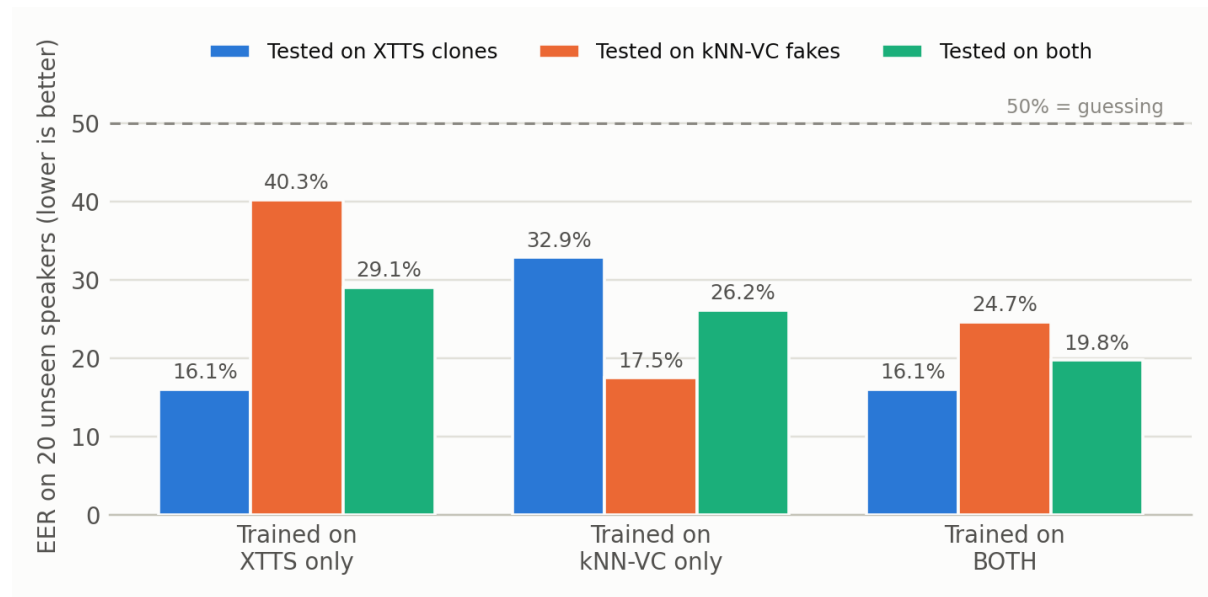


Figure 1. EER on 20 unseen LibriSpeech speakers for heads trained on XTTS only, kNN-VC only, or both. All audio phone-line (G.711).

Trained on	EER vs XTTS	EER vs kNN-VC	EER vs both	XTTS caught	kNN-VC caught	Real flagged
XTTS only	16.1%	40.3%	29.1%	79.8%	27.4%	10.3%
kNN-VC only	32.9%	17.5%	26.2%	37.3%	76.9%	13.4%
Both	16.1%	24.7%	19.8%	88.7%	76.0%	25.4%

Strict silence-only EER: 46.3% (2,123 clips with long enough pauses), so there is no background shortcut behind these numbers.

- **The failure runs both ways:** XTTS-only scores 40.3% on kNN-VC; kNN-VC-only scores 32.9% on XTTS.
- **Adding a second generator costs nothing on the first:** XTTS stays at 16.1%. It cuts kNN-VC from 40.3% to 24.7% and the combined EER from 29.1% to 19.8%.
- **Generator variety matters, data volume doesn't:** 70 vs 5 training speakers made no difference to the unseen-generator EER; a second generator did.

Caveats. (1) At its default threshold the two-generator head flags 25.4% of real voices, against 10–13% for the single-generator heads; the threshold needs tuning for deployment. (2) Once kNN-VC is in training it is no longer an unseen test for this head; the honest claim needs a third generator kept out entirely.

3. Which layers each head listens to

Each head learns its own softmax weight over the 13 encoder layers. Although both read identical features, they settled on **different layers**:

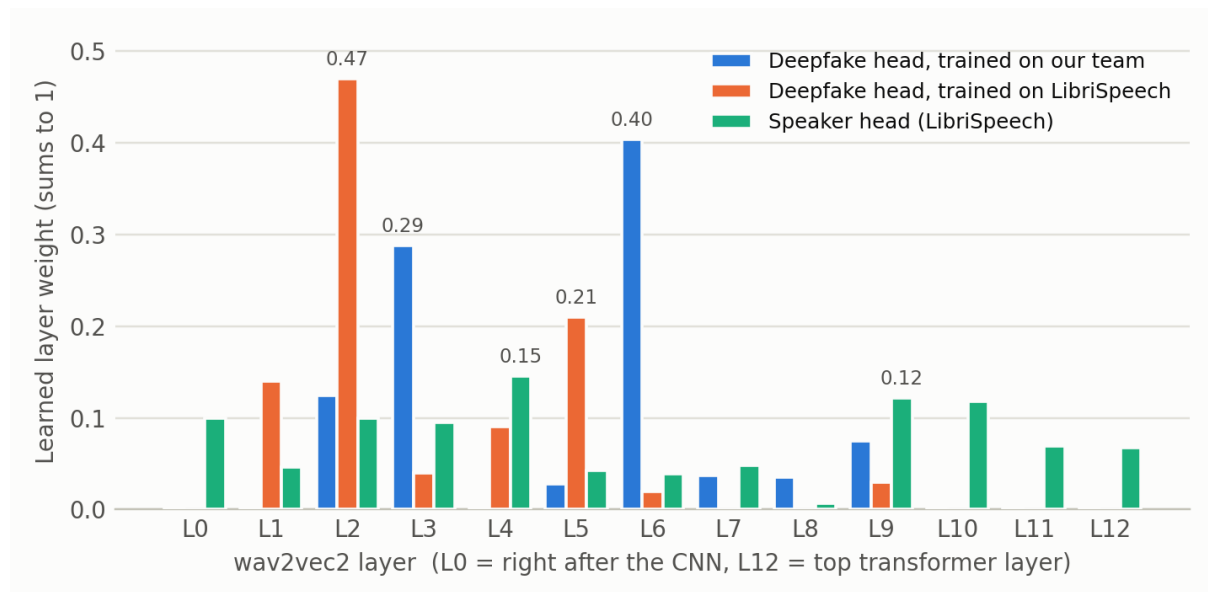


Figure 2. Learned layer weights. Deepfake head: team model (final recipe, averaged over the six fold heads) and LibriSpeech model (phone-line). Speaker head: the LibriSpeech-trained model used in the app.

- **Deepfake head: low and middle layers.** On our team it relies on L6 (0.40), L3 (0.29) and L2 (0.12); on LibriSpeech on L2 (0.47), L5 (0.21) and L1 (0.14). The top layers L10–L12 get almost no weight in either model (0.002 or less on the team, under 0.01 on LibriSpeech).
- **Speaker head: spread out.** No layer gets more than 0.15. The largest shares are L4 (0.15), L9 and L10 (0.12 each), and L0 and L2 (0.10 each); L8 gets almost nothing (0.007).
- **Stable across recipes.** In all three team runs (weights averaged over the six fold heads) the same three layers carry most of the deepfake head's weight: L2 + L3 + L6 = 69.5%, 74.8% and 81.6%.

Team run	Top deepfake layers (weight)	L2 + L3 + L6
1: noise only	L2 0.29 · L6 0.26 · L3 0.14 · L9 0.12	69.5%
2: + pause gating	L3 0.36 · L6 0.25 · L2 0.14 · L7 0.10	74.8%
3: + pitch/speed (final)	L6 0.40 · L3 0.29 · L2 0.12 · L9 0.08	81.6%

Interpretation. Lower layers of wav2vec2 stay close to the waveform and keep fine acoustic detail, which is where vocoder and synthesis artifacts live. Middle layers carry more phonetic information. Speaker identity is spread over several levels, from timbre near the bottom to higher layers. This matches published layer-wise analyses of wav2vec2 (for example Pasad et al., 2021). It is also the argument for one encoder with two heads: each head takes what it needs from the same pass, with no separate speaker network.

4. Speaker head and voiceprints

4.1 Training on LibriSpeech

Held-out speaker EER went **9.47% → 6.21% → 3.96%**. The first two numbers come from a 40-speaker pilot (28 training speakers, 12 held out). The first model memorised its training speakers (100% training accuracy). Mini-batches, early stopping, and a phone-line copy of every clip under the same label brought it to 6.21%. Training on train-clean-100 (211 speakers, 40 held out) gave the final 3.96%. Because every

clip is used both clean and through the phone line, the voiceprint learns to ignore the channel. On LibriSpeech, different speakers sit near zero similarity (cosine ≈ 0.006), the same speaker saying different sentences at ≈ 0.63 , and the same clip clean vs phone-line at ≈ 0.87 .

4.2 On our team: domain shift

On our six teammates, never seen in training, speaker EER is about **11%**. The same person scores 0.84–0.85 against their own voiceprint, but **different teammates score 0.60–0.62**, far above LibriSpeech's ≈ 0.0 . The head learned to separate English audiobook readers, not Indian-accented phone speech, and our team shares phones, rooms and accent. The decision threshold therefore had to rise from LibriSpeech's 0.319 to about 0.75. That threshold is the equal-error point of the same team trials the EER is measured on, so these team numbers are slightly optimistic.

4.3 How to build a voiceprint: mic, codec, or both

Each voiceprint was built from sentences 001–010 in one of three ways, then tested on sentences 011–030 heard the same way:

Voiceprint built from	Tested on	EER	Threshold	Similarity: same / other / clone	Gap	XTTS clones pass
codec (phone line)	codec	10.9%	0.741	0.85 / 0.60 / 0.70	0.24	43.3%
mic (clean)	mic	11.7%	0.759	0.85 / 0.62 / 0.64	0.23	14.2%
average of both	both, pooled	12.1%	0.740	0.84 / 0.60 / 0.67	0.23	30.4%

Gap = similarity to the same person minus similarity to other teammates. 120 genuine and 600 impostor trials per row (twice that for the pooled row, which tests both channels); differences under about 2 points are noise.

All three are a **tie** on telling people apart (10.9–12.1% EER, gap 0.23–0.24), so switching to phone-line voiceprints costs nothing. We use **codec-only voiceprints**, which match how calls actually sound. Every voiceprint variant we ran landed at about 10–12% team EER, so a point either way is run-to-run noise.

4.4 Over a phone line, clones fool the voice check three times as often

With matched audio, XTTS clones pass the voice check **14%** of the time on a clean mic but **43%** over a phone line. Most likely the codec removes high-frequency detail (above 4 kHz) that set the clones apart. kNN-VC fakes, scored as clean audio against the phone-line voiceprints, pass **71%** of the time. Pass rates also vary a lot by person: in one run, 45% of Asmit's clones passed but none of Nikhil's or Yogya's (Appendix A.3).

Why both heads are needed: over a real call a voice check alone lets almost half of XTTS clones and most voice-conversion fakes through. The deepfake head catches 94.7% of XTTS clones on the same kind of audio.

5. Live-system design decisions

5.1 The pipeline, every 3 seconds

1. The browser sends microphone audio in 0.5 s pieces; a streaming resampler converts them to 16 kHz without seams.
2. Every 3 s the latest window is checked by an energy voice-activity detector; windows with under 40% speech are skipped.
3. The window goes through the phone line (8 kHz G.711) when the call is a phone call, including the live microphone mode.
4. One wav2vec2 pass; both heads read the same 13 layers.

- The deepfake score is the median of the last three windows; the voiceprint is a running average; the two are fused into one risk, which is smoothed over time.

Voice-activity detection. Each 3 s window is cut into 150 slices of 20 ms. A slice counts as speech if it is at least 10 dB louder than the window's own background (its quietest 10% of slices) and louder than -60 dBFS. Measuring against each window's own background works at any volume.

5.2 Median of the last three windows

One odd 3 s window (a laugh, a shout, a burst of noise) should not turn a genuine call red. The deepfake score used for the risk is the **median of the last three windows**: one outlier is outvoted, two in a row count. A clone that sounds fake throughout is still caught at the first window, because there is nothing to outvote.

Scenario	Risk over the call
Genuine caller, one "shocked" window at 12 s	PASS throughout (risk stays between 5 and 22)
AI clone for the whole call	HOLD from the first window (3 s)
Genuine caller, a clone takes over at 6 s	PASS to 9 s, VERIFY at 12 s, HOLD at 15 s
Another real person claiming to be the caller	PASS to 6 s, VERIFY at 9 s, HOLD at 12 s
Same, with a very different voice (similarity 0.30)	HOLD at 6 s, once 4 s of speech are in

Logic tests, not recordings: scripted scores fed through the live scoring code. Genuine windows score 0.05 synthetic and cloned windows 0.95; voice similarity is 0.85 for the genuine caller, 0.70 for the clone and 0.60 for another person (our team's measured averages); 85% of each window is speech.

5.3 How the two heads are fused

Risk is the probability that the call is **synthetic OR not the person the caller ID claims**: $1 - (1 - p_{\text{synthetic}})(1 - p_{\text{not_them}})$, shifted by a call-context prior and smoothed from window to window. The speaker part turns on only after 4 s of speech. Bands: **PASS** below 45, **VERIFY** 45–75, **HOLD** 75 and above.

An earlier version added the two scores together in log-odds. It failed a key case: a real human pretending to be someone else looked strongly "not synthetic", which cancelled out the voice mismatch, so the call stayed at PASS. With the OR rule the same call reaches VERIFY at 9 s and HOLD at 12 s (table above). A voice match can never lower the risk; it can only leave the synthetic score to speak for itself. Context raises the prior: unknown number +0.7, asks for money or an OTP +1.0, urgent or secret request +0.5 (log-odds). Today these context flags are set by hand in the app; the scam-language model in section 8 will set them from what the caller actually says.

5.4 Silence holds the score

When nobody speaks, no window is scored and the risk stays where it was. **Silence is not evidence either way**, and it also means an attacker can't go quiet to cool the alarm down. We tested a version where each silent window halved the distance back to a calm baseline, so an alarm faded to PASS within about 6–12 s of silence. We rejected it for that reason.

5.5 Phone line everywhere

- Training:** every clip has a phone-line copy, and the LibriSpeech experiments are phone-line only.
- Live microphone:** each window goes through G.711 before scoring. Before this, mic tests ran on clean audio that no real call would produce.
- Voiceprints:** built from phone-line audio (section 4.3).

5.6 Seamless live audio

Converting each 0.5 s microphone piece separately put a small discontinuity at every seam, twice a second: quiet on average but sharp, exactly the kind of high-frequency artifact a deepfake detector reacts

to. A streaming resampler now converts the live stream exactly as if it were one recording (identical to floating-point precision). It keeps about 1 s of buffer and also ignores repeated audio if the browser resends the whole recording.

5.7 An honest demo

Each teammate's demo call is scored by the fold head that **never heard that teammate**. The head trained on all six has seen every teammate's sentences, so using it on them would grade it on its own training data. New voices (uploads, the live microphone) use the head trained on everyone.

5.8 Speed, footprint and privacy

Measure	Value
Per 3 s window (median / 95th percentile)	18.3 ms / 30.4 ms
... of which wav2vec2 encoder / both heads	15.9 ms / 2.4 ms
Speed	164× faster than real time
Peak GPU memory	0.28 GB
Live microphone audio	A rolling buffer of about 4 s, in memory only; our code never saves it
Stored per enrolled person	One 192-number voiceprint (768 bytes); no audio

6. Limitations

- **Six speakers.** Each per-speaker result rests on one person's 30 real and 30 cloned sentences (about 100 real and 130 cloned windows), so the $\pm$ values are wide ($4.6\% \pm 3.3\%$).
- **Unseen generators.** The app's deepfake head is trained on XTTS only and drops to about 40% EER on an unseen generator. The two-generator recipe is not yet in the app, and with kNN-VC in its training it needs a third generator for a truly unseen test.
- **False alarms.** The two-generator head flags 25.4% of real voices at its default threshold.
- **Speaker head domain.** It was trained on English audiobooks; on Indian-accented phone speech EER is about 11%, and the threshold was tuned on the same team data.
- **Emotional speech.** Training covers pitch changes of about -1.8 to $+1.7$ semitones; shouting, laughing or crying are larger changes and are not in the data.
- **Voice-activity detection** is energy-based, so music, a TV or a cough can count as speech.
- **kNN-VC fakes** were built from only about 80 s of each target's audio (the method does better with several minutes), so their quality is moderate.

7. Next steps

- Finish the **scam-language score** and the **pitch and prosody analysis**, and add both to the risk score (section 8).
- Test the two-generator head on a **third generator kept out of training** (OpenVoice, or IndicF5 for Hindi and Tamil).
- Move the **two-generator recipe into the app's deepfake head**, and tune its threshold for fewer false alarms.
- Train the speaker head on **Indian speakers**: NISP (345 speakers, English plus five Indian languages) and Kathbath (1,218 speakers, 12 languages), keeping Svarah (Indian-accented English) as an untouched test set.
- Measure **false alarms on real speech from other sources** (NISP, Svarah) with the deepfake head.

- Record and clone **expressive speech** (shocked, excited, rushed, angry); stress-test with babble noise, AMR and Opus codecs and packet loss; replace the energy VAD with a **neural VAD** (Silero).

8. Work in progress: two more signals for the risk score

A clone that is good enough could pass both heads, so we are adding two more signals that look at the call from different angles. **Both are in progress: there are no results for them yet, and nothing in sections 1 to 7 depends on them.**

8.1 Scam-language score (our "sentiment score")

What it is. A speech-to-text step turns the call into words, and a text model scores how likely those sentences are to come from a fraudster: urgency ("right now"), secrecy ("don't tell anyone"), requests for money, UPI transfers or OTPs, pressure, and claims of authority ("I'm calling from your bank"). We are starting with English and Hindi, the two languages in our team data.

Why it helps. A cloned voice can sound exactly right, but a scammer still has to ask for the money or the OTP. What is said is the one part of the call the fraudster cannot avoid.

How it will feed the risk score. Today a person ticks three call-context options by hand (section 5.3). The scam-language score will set that context automatically from what the caller actually says. Like the other signals, it can only raise the risk, and the words will be scored on the fly and not stored.

8.2 Pitch and prosody analysis

What it is. Prosody is the melody and rhythm of speech: how pitch rises and falls, how fast someone talks, where they pause and how loudness changes. This part tracks those patterns across the call.

Why it helps. Cloning tools copy the sound of a voice well but can get its movement wrong: too flat, too regular, or emotion that doesn't fit the words. It also helps the other way: a genuinely scared caller's real jumps in pitch and speed should read as human, not suspicious. This is different from section 2.4, which teaches the deepfake head to ignore a simple pitch shift; here we look at how pitch, timing and loudness move over the whole call.

How it will feed the risk score. As a third signal next to the two heads, with the same rule: any signal can raise the risk, and none can lower what another has found.

8.3 All the signals together

Signal	Question it answers	Status
Deepfake head	Is this voice made by AI?	Working (sections 2 and 3)
Speaker head	Is this the person on the caller ID?	Working (section 4)
Scam-language score	Does what they're saying sound like a scam?	In progress (8.1)
Pitch and prosody	Does the voice move like a real person's?	In progress (8.2)

Appendix A. Per-speaker results

A.1 Deepfake head, final recipe (6-fold leave-one-speaker-out)

Held-out speaker	EER	Phone	Clean	Real flagged	Fakes caught	Silence-only
Saksham	3.6%	6.5%	1.6%	7.0%	98.5%	36.7%
Asmit	1.7%	3.4%	0.0%	9.8%	100.0%	38.3%
Vijay	11.9%	5.3%	18.4%	6.4%	78.6%	33.3%
Nikhil	3.8%	2.8%	2.8%	9.8%	99.2%	46.7%
Yogya	2.8%	4.1%	3.9%	5.8%	98.0%	35.0%

Held-out speaker	EER	Phone	Clean	Real flagged	Fakes caught	Silence-only
Shreyas	3.9%	6.0%	3.5%	0.0%	84.8%	35.0%
Mean	4.6% ± 3.3%	4.7%		7.1%*	94.7%*	37.5%

Per-speaker flagged/caught use the raw 0.5 threshold; * = after probability calibration on out-of-fold scores.

A.2 Unseen generator on the team (heads trained on XTTS only)

Speaker (voice converted from)	EER vs XTTS	EER vs kNN-VC	kNN-VC caught	kNN-VC pass voice check
Saksham (from Asmit)	4.1%	52.8%	13.2%	80%
Asmit (from Vijay)	1.3%	33.6%	26.8%	65%
Vijay (from Nikhil)	8.2%	36.1%	6.0%	75%
Nikhil (from Yogya)	3.5%	18.5%	20.0%	80%
Yogya (from Shreyas)	2.9%	37.5%	11.6%	35%
Shreyas (from Saksham)	4.6%	63.6%	0.0%	90%
Mean	4.1%	40.3% ± 14.4%	12.9%	71%

Each fake uses the next teammate's sentences 011–030 in this speaker's voice. Voice check: kNN-VC fakes as clean audio against the phone-line voiceprints.

A.3 Voice check vs XTTS clones, per person

Speaker	Same-person similarity	Clone similarity	Clones passing (clean voiceprint)	Clones passing (voiceprint with phone-line copies)
Saksham	0.84	0.69	25%	25%
Asmit	0.84	0.72	30%	45%
Vijay	0.86	0.57	5%	15%
Nikhil	0.87	0.63	0%	0%
Yogya	0.83	0.57	0%	0%
Shreyas	0.85	0.66	20%	30%
All	0.85	0.64	13% (phone line 20%)	19% (phone line 33%)

Similarities and the first pass-rate column come from the first voiceprint run (clean voiceprints); the last column from a later run whose voiceprints also included phone-line copies. Each person has 20 test clips, so one clip = 5%.

Appendix B. Glossary

EER: Equal error rate: the error at the threshold where false alarms equal misses. Lower is better; 50% is guessing.

Leave-one-speaker-out: Train on all speakers but one, test on the one left out, and repeat for every speaker.

Fold head: The model trained with one speaker left out; used to score that speaker honestly.

wav2vec2: A speech model pretrained on unlabelled audio; we use its 13 internal layers as features and never retrain it.

XTTS-v2: A text-to-speech voice-cloning model: text plus a short sample of a voice produces speech in that voice.

kNN-VC: A voice-conversion model: someone's speech is re-voiced into a target speaker using nearest-neighbour matching of WavLM features.

G.711 μ-law: The classic telephone codec: 8 kHz sampling, nothing above 4 kHz, 8-bit companded samples.

SNR: Signal-to-noise ratio in dB; 30 dB is faint background noise, 5 dB is noise nearly as loud as the voice.

Platt scaling: Fitting $p = \text{sigmoid}(a \cdot \text{score} + b)$ on held-out scores, so a score becomes a calibrated probability.

AAM-softmax: Additive angular margin loss; trains voiceprints of the same speaker into a tight cone.

Voiceprint / cosine similarity: A 192-number summary of a voice; two voiceprints are compared by the cosine of the angle between them (1 = identical direction).

VAD: Voice-activity detection: deciding which parts of the audio contain speech.